

Alunos: Amanda Jury Nakamura e Carlos Eduardo Correa Zanon

Disciplina: Sistemas Operacionais **Professor:** Marco Aurélio Wehrmeister

Relatório Projeto B - Simulação de Sincronização, Concorrência e E/S

Arquivos Submetidos

O projeto é uma evolução do simulador desenvolvido em C++ com o framework Qt na parte A. Os seguintes arquivos estão sendo submetidos:

- **Escalonador.cpp e Escalonador.h:** Lógica do motor de escalonamento, com suporte a envelhecimento (PRIOPEnv), travas de exclusão mútua (Mutex) e suspensão por interrupções físicas (E/S).
- **TCB.h e TCB.cpp:** O Bloco de Controle de Processos, com melhorias para rastrear `prioridade_dinamica`, `io_restante`, `mutex_esperado` e a fila de eventos.
- **Interface.cpp e Interface.h:** Renderizador visual atualizado para mapear padrões de preenchimento e eventos de hardware em uma nova área de tarefas suspensas.
- **main.cpp e projeto.pro:** Inicializador e arquivo de configuração de compilação automática para o qmake.
- **Arquivos .txt:** Cenários de teste contemplando concorrência de Mutex e operações de E/S.

Telas do Simulador e Elementos Visuais

1. Tela Inicial e Nova Legenda

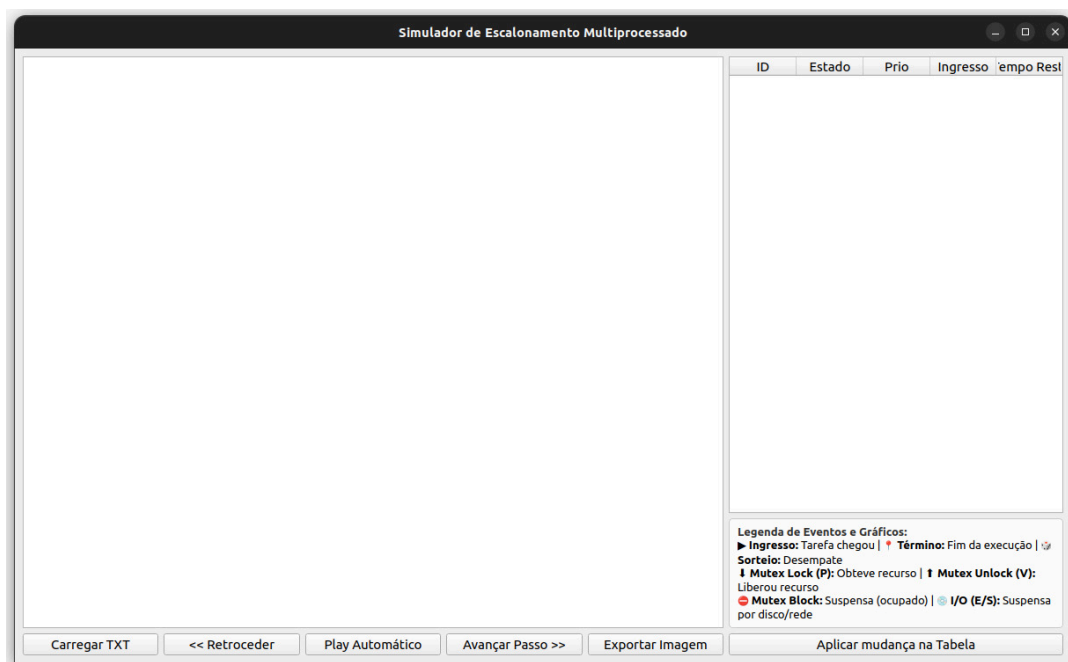


Imagem 01: Tela inicial vazia, com o novo painel de legendas.

Elementos Visuais

- **Painel de Legenda (Canto Inferior Direito):** Um novo componente fixo que atua como um dicionário visual para os novos eventos do Projeto B. Ele mapeia os símbolos (badges) e os padrões de hachura do Gráfico de Gantt, facilitando a leitura imediata das operações de E/S e Mutex durante a simulação.

2. Tela da simulação em andamento

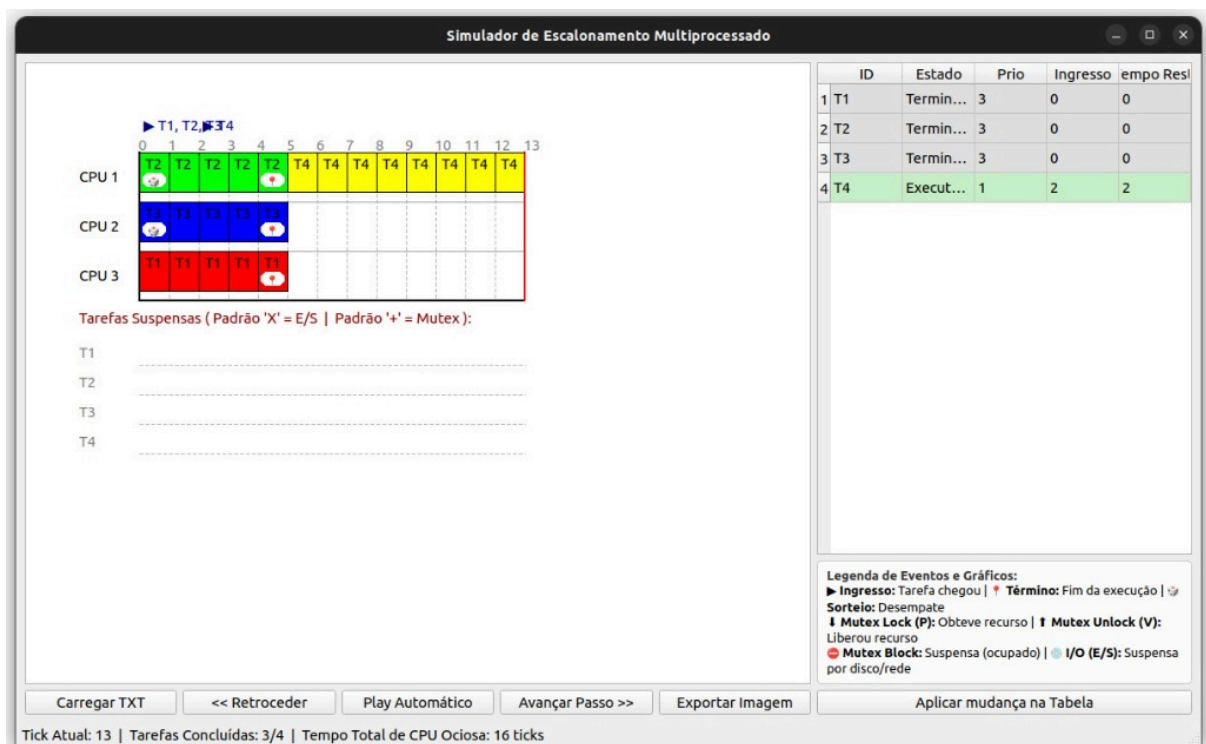


Imagem 02: Simulação do particionamento do Gráfico de Gantt em CPUs e Tarefas Suspensas.

Elementos Visuais e Origem dos Dados:

- **Área Superior (CPUs):** Exibe as tarefas no estado EXECUTANDO. A informação provém do vetor cpus do motor, onde blocos coloridos são desenhados de acordo com o id da tarefa que ocupa cada processador no instante registrado pela Snapshot.
- **Área Inferior (Tarefas Suspensas):** Nova região do gráfico reservada exclusivamente para tarefas que perderam o processador de forma não-voluntária ou voluntária. É renderizada varrendo a lista_tarefas do Snapshot em busca de processos cujo status seja Estado::SUSPENSA.
- **Padrão Quadriculado Diagonal ('X'):** Representa visualmente uma tarefa bloqueada por uma operação de Entrada e Saída (E/S). O Qt aplica o

Qt::DiagCrossPattern sempre que a variável `t.io_restante` da tarefa for maior que 0.

- **Padrão Quadriculado Cruzado ('+')**: Representa uma tarefa bloqueada aguardando a liberação de uma região crítica (Mutex). O Qt aplica o Qt::CrossPattern sempre que a variável `t.mutex_esperado` for diferente de -1.

3. Ícones de detalhamento dos Eventos

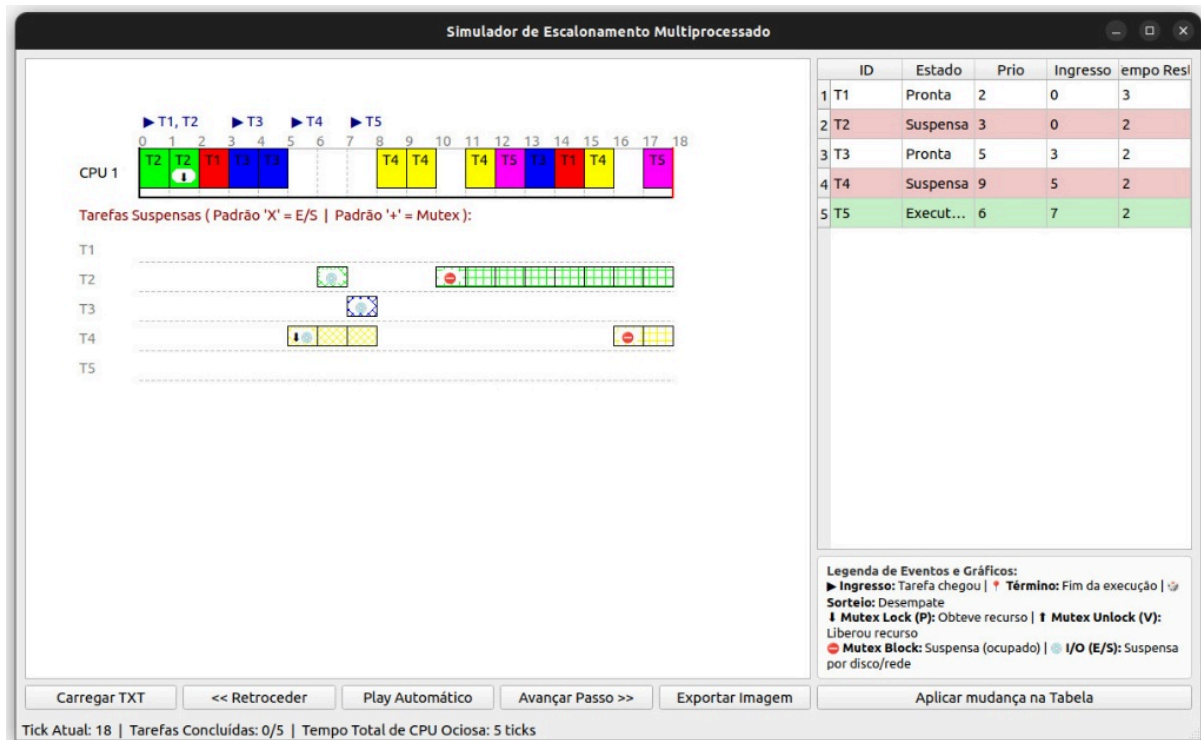


Imagem 03: Foco nos badges (ícones circulares) que detalham o comportamento assíncrono do sistema.

Para evitar sobreposição de texto, os eventos que ocorrem em um milissegundo exato são agrupados no método `drawBadge` e desenhados dentro de um pequeno círculo branco (com Z-value elevado para flutuar sobre os blocos). As informações vêm do mapeamento de eventos (`eventos_neste_tick`):

- **Ingresso:** A tarefa entrou no sistema.
 - Lógica: `tempo_ingresso == clock_global`.
- **Término:** A tarefa foi finalizada.
 - Lógica: `tempo_restante == 0`.
- **Sorteio:** A tarefa venceu uma disputa de CPU através do critério de desempate randômico (`std::rand`).
- **Mutex Lock (P):** A tarefa executou a instrução ML e conseguiu obter a trava com sucesso, registrando seu ID no `std::map<int, int>` `mutexes` do Escalonador.

-  **Mutex Unlock (V):** A tarefa executou a instrução MU, liberando o recurso (alterando o mapa de mutexes para -1) e acordando as tarefas bloqueadas.
-  **Mutex Block:** A tarefa executou ML, mas o mapa de mutexes indicou que o recurso já pertencia a outra tarefa. O bloco é interrompido imediatamente.
-  **I/O (E/S):** A tarefa iniciou uma operação de disco/rede (IO), suspendendo-se e ativando o contador de hardware (io_restante).

4. Tabela de Estados e Relógio Global

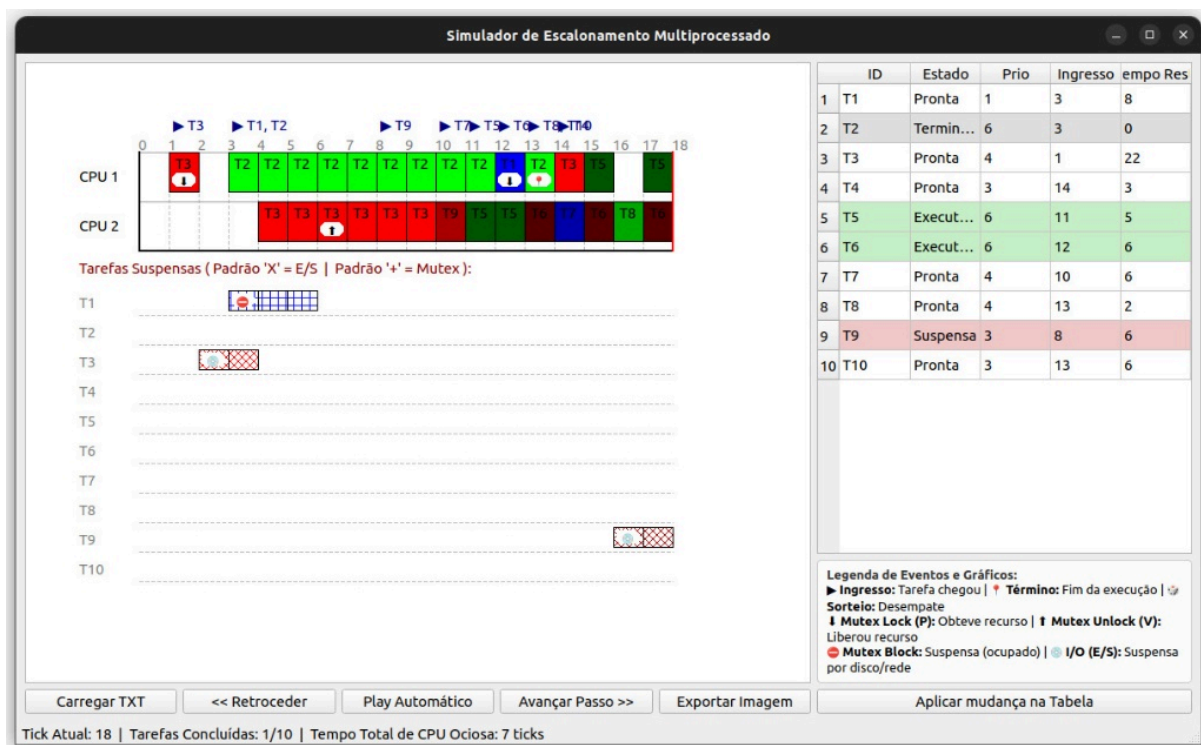


Imagem 04: Acompanhamento matricial e status do sistema.

Elementos Visuais e Origem dos Dados:

- **Tabela de Tarefas (Painel Direito):** Atualiza em tempo real os metadados das tarefas. A informação é lida diretamente do array `lista_tarefas`. As linhas mudam a cor de fundo dinamicamente: Verde para EXECUTANDO, Vermelho/Rosa claro para SUSPENSA (indicando o bloqueio por E/S ou Mutex), Cinza para TERMINADA e Branco para tarefas prontas ou novas.
- **Barra de Status Inferior:** O Tick Atual reflete a variável `clock_global`, Tarefas Concluídas reflete o contador `tarefas_concluidas`, e o Tempo Total de CPU Ociosa espelha a variável `tempo_ocioso` (incrementada sempre que uma CPU recebe `nullptr` enquanto há tarefas incompletas no sistema).

5. Exportação de Histórico (Imagem PNG)

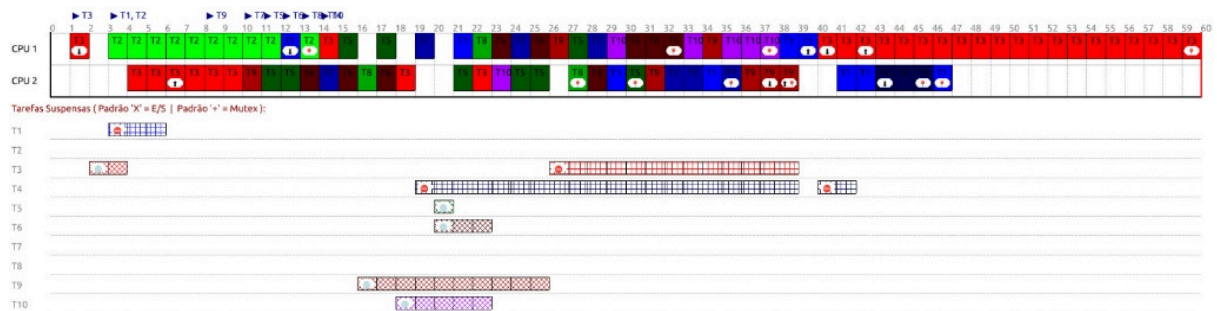


Imagem 05: Exportação do cenário de execução.

Elementos Visuais e Origem dos Dados:

- **Canvas de Gantt:** Utilizando a função `btnExportarClicado`, o sistema renderiza todo o histórico acumulado no vetor de coleção de Snapshots em um arquivo PNG. Isso permite a auditoria completa da simulação, provando visualmente o funcionamento das interrupções de hardware (E/S) e a liberação em cadeia de Mutexes que estavam segurando o fluxo de execução das tarefas ao longo do tempo.